UNIVERSITY OF CENTRAL FLORIDA

A Custom Gait Controller for the Unitree Go1 Robot Dog

EEL 5669 Autonomous Robotic Systems

Fall 2025 Class

12/08/2025

Abraham Alcala, Steven Booth, Marlon Garcia, Thomas Greenwood, Tristan Haack, Jordan Haag, Nihad Habizada

ABSTRACT

This project explores the implementation of biologically inspired gaits on the Unitree Go1 robot, focusing on the walk, trot, canter, and gallop gaits. Quadrupedal locomotion requires coordinated timing of swing, stance, and transition phases across four independent legs. Each gait was analyzed in terms of duty cycle, phase offsets, and limb coordination to create a programmable framework for generating stable and efficient robotic motion. This report documents the gait modeling process, presents timing diagrams for each locomotion pattern, and establishes the structural foundation for trajectory generation, control system integration, and real-world deployment of the Go 1 robot. In the lab, we achieved all four gaits that are capable of walking without falling over. The walk gait also features an IMU-based yaw-correction controller, which keeps the robot walking into a new line.

I. INTRODUCTION

Quadrupedal robots have emerged as a leading platform in legged locomotion research due to their ability to traverse uneven terrain, maintain stability under disturbance, and replicate the dynamic motion patterns of biological animals. Among these platforms, the Unitree Go1 robot provides a compact, highly actuated system capable of executing a wide range of locomotion behaviors.

This project focuses on modeling and implementing four common quadrupedal gaits such as walk, trot, canter, and gallop. By utilizing duty cycles and phase offsets to describe each gait's footfall sequence, specific per gait timing patterns can be established. These timing patterns are the foundation for trajectory generation and future control implementation on both the simulated and physical Go1 robot. The goal is to enable the robot to perform stable motions as well as more dynamic behaviors by following structured and repeatable gait timing diagrams.

II. BACKGROUND

The Unitree Go1 is a high-performance quadrupedal robotic platform designed to emulate the locomotion of four-legged animals. Its four identical legs, each containing three degrees of freedom, allow for a wide range of gait patterns that vary in stability, efficiency, and achievable speed. Understanding the underlying timing and footfall characteristics of these gaits is crucial to designing controllers capable of coordinated locomotion.

The term gait is defined as a sequence of motion the legs of a walking object perform over time to achieve a change in position.[1] Quadruped robots have similar kinematics to that of canines. It is important to study the different gait cycles that canines can exhibit so that proper timing and ultimately trajectory can be generated for each limb.[2] The four gait cycles that will be implemented with the Unitree Go1 robot are walk, trot, gallop, and canter gaits.[2] More detailed discussion for each gait will be discussed in future sections.

III. WALK GAIT

The walk gait is the simplest out of the four gaits. This gait is considered a static gait because at any given time, the stance intervals of three legs overlap providing an increase in stability compared to other gaits.[3] In figure 1, the phase diagram for the walk gait can be seen. When the pulse equals logic 1, the leg is in the swing phase of the gait cycle. A logic 0 represents the stance phase. Looking at figure 1, at any given time, there are three legs in the stance phase of the walk gait.

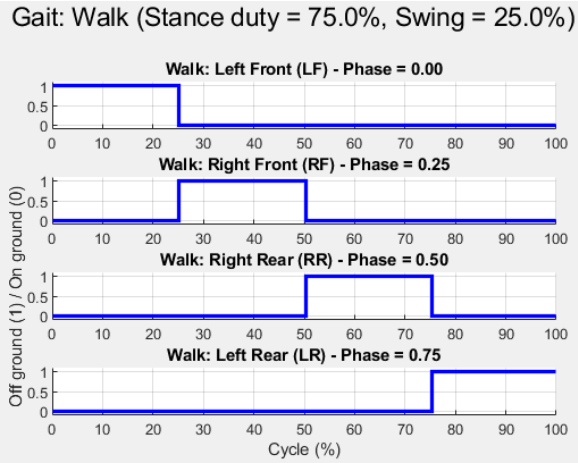


Figure 1: Walk Gait Phase Diagram

IV. TROT GAIT

The trot gait is considered a dynamic 2 beat gait with diagonal forward and hind leg pairing.[1] In figure 2, the LF and RR legs are in phase with one another and spend 50% of the gait cycle in the swing phase. Also, the RF and LR legs are paired but have a 50% phase offset compared to the other leg pair.



Figure 2: Trot Gait Phase Diagram

V. GALLOP GAIT

Ther gallop gait is an asymmetric dynamic gait that has forward, and rear legs paired with phase offset.[1] This gait produces four distinct beats. Notice in figure 3 there the LF and RF have a phase offset of 20% and the LR and RR have a phase offset of 20%. Also, observe that there is one period equating to about 20% of the total gait cycle time that all four legs are off the ground.

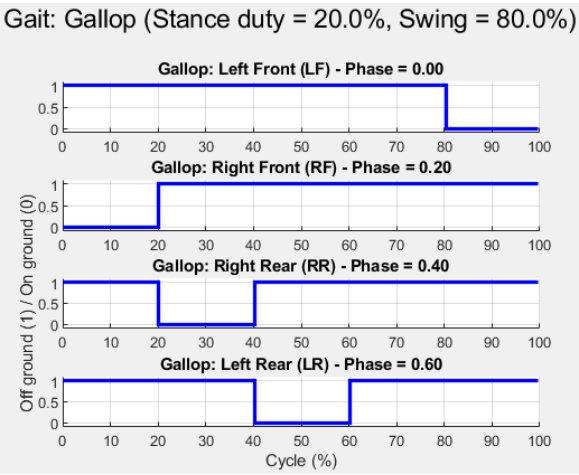


Figure 3: Gallop Gait Phase Diagram

VI. CANTER GAIT

The canter gait is an asymmetric dynamic gait that has at times 1, 2, and 3 legs on the ground at a given time. Also notice how each leg has different stance and swing phases compared to the other gaits. This a

different compared to the previous three gaits discussed. As a result,

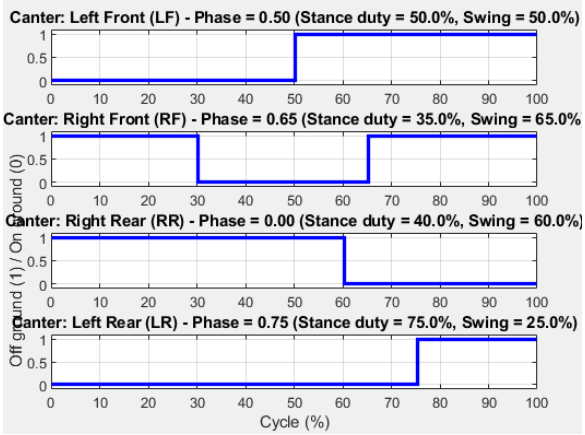


Figure 4: Canter Gait Phase Diagram

VII. KINEMATICS

The Unitree Go1 leg can be modeled as a three degree of freedom serial manipulator consisting of a hip abduction (Joint 1), a hip pitch joint (Joint 2), and a knee pitch joint (Joint 3). All four legs share identical geometry, differing only by their mounting positions on the robot's body. Kinematic modeling is required to convert task-space foot trajectories into joint-space commands suitable for execution on the robot.

A. Leg Geometry and Coordinate Frames

Each leg consists of a hip offset, a thigh link, and a shank link. Using the geometry provided in manufacturer CAD data, the relevant dimensions are:

- Hip vertical offset: $d_1 = 60.1\,mm$
- Hip horizontal offset: $d_2 = 81\,mm$
- Thigh link length: $a_2 = 213\,mm$
- Shank link length: $a_3 = 212.3\,mm$
- Foor radius: $r_f = 22.9\,mm$

Coordinate frames are defined using the Denavit-Hartenberg convention. Join 1 rotates about a vertical axis, generating lateral foot motion. Joints 2 and 3 rotate in the sagittal plane, forming a planar 2-link chain responsible for forward and backward and vertical foot motion.

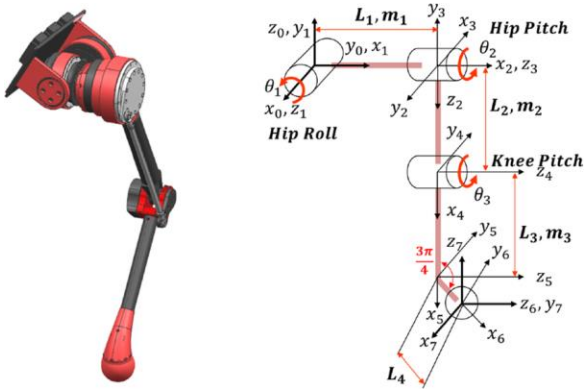


Figure 5: Coordinate frames and link geometry for a Go1 leg

B. DH Parameterization

The leg's geometry is captured using the following standard DH table:

Link	a_i	α_i	d_i	θ_i
1	0	90°	d_1	θ_1
2	a_2	0	d_2	θ_2
3	a_3	0	0	θ_3

Table 1: DH Table for Go1 leg

The forward kinematic transformation of the DH table above provides the ED position of the foot relative to the hip joint frame.

$$T_3^0 = \begin{bmatrix} c_1\,c_{23} - s_1\,s_{23} & -(c_1\,s_{23} + s_1\,c_{23}) & 0 & c_1\,(a_2\,c_2 + a_3\,c_{23}) \\ s_1\,c_{23} - c_1\,s_{23} & -(s_1\,s_{23} + c_1\,c_{23}) & 0 & s_1\,(a_2\,c_2 + a_3\,c_{23}) \\ 0 & -1 & 0 & d_1 + d_2 + a_2\,s_2 + a_3\,s_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 6: Forward Kinematic Transformation

C. Forward Kinematics

Forward kinematics compute the foot position (x_c, y_c, z_c) given joint angles $(\theta_1, \theta_2, \theta_3)$. Because Joints 2 and 3 move in a plane rotated by Joint 1, the foot position is:

- $x_c = d_1 + a_2 cos\theta_2 + a_3 \cos(\theta_2 + \theta_3)$
- $r_c = a_2 sin\theta_2 + a_3 \sin(\theta_2 + \theta_3) + d_2$
- $y_c = r_c cos\theta_1$
- $z_c = r_c sin\theta_1$

This mapping is essential for validating reachable workspace and enforcing leg joint limits.

D. Inverse Kinematics

Inverse Kinematics determine the joint angles required to place the foot at a desired position. For the Go1, inverse kinematics separates cleanly into:

1. Hip Abduction/Adduction (Joint 1)

Given the foot's lateral position (y_c, z_c), the hip angle is:

- $\phi = \arctan 2(y_c, z_c)$
- $r_2 = \sqrt{y_c^2 + z_c^2}$
- $\alpha = \arctan 2\left(d_2, r_2 - \frac{d_2^2}{r_2}\right)$
- $\theta_1 = \phi + \alpha$

This resolves the leg's orientation about the vertical axis, accounting for the hip offset.

2. Hip Pitch and Knee Pitch (Joints 2 and 3)

The sagittal-plane inverse kinematic is treated as a 2-link manipulator. Let

$$s = x_c - d_1,\ L = \sqrt{s^2 + (r_2 - d_2)^2}$$

Using the law of cosines, the knee angle is:

$$\theta_3 = \arccos\left(\frac{L^2 - a_2^2 - a_3^2}{2a_2a_3}\right)$$

The hip pitch angle is:

$$\theta_2 = \arctan 2(r_2 - d_2,\ s) - \arctan 2(a_3 \sin\theta_3,\ a_2 + a_3\cos\theta_3)$$

These equations provide a unique physical solution corresponding to normal walking posture.

VIII. FOOT TRAJECTORY

A complete understanding of each gait's stance and swing phases are needed to generate foot trajectories. The first two parameters needed are the robot's COM velocity and the gait cycle time. From these, the length of the stance phase can be derived for each leg under the condition the COM velocity is constant. A cubic spline was utilized to generate a smooth swing path for each gait. Appropriately spaced x and z knots are required to generate the spline so that the beginning and end of the stance phase as well as the maximum design height of the end effector can be achieved by the end effector. Figure 7 shows the paths generated for each gait, as well as the maximum reach of the end effector based on robot joint limits.

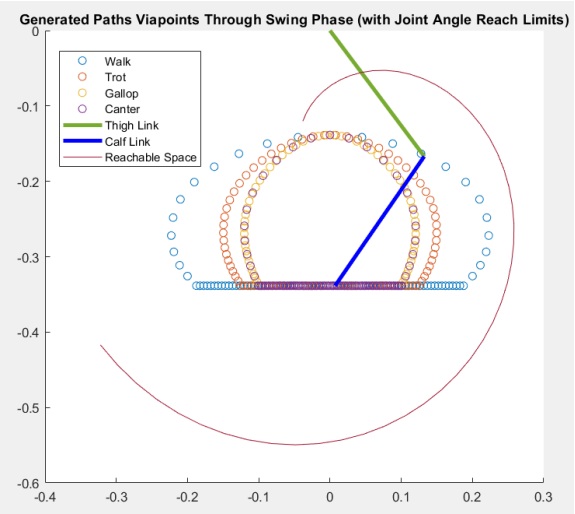


Figure 7: Generated Paths for Each Gait

The total trajectory is a combination of the path of the swing and stance phase that are evenly parameterized with respect to the total gait cycle time. Inverse kinematics are then applied to each (x,z) coordinate so that the desired $\theta_{1,des}$, $\theta_{2,des}$ and $\theta_{3,des}$ joint angles are generated. The desired joint angle arrays are then circularly shifted to achieve appropriate per leg timing. This shifting is based off the phase values for each leg. The following blocks depict the main sections of the gait generation code.

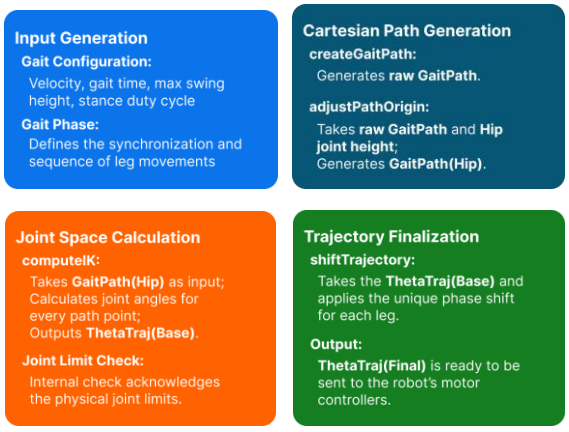


Figure 8: Main sections of the gait generation code

IX. CODING

In this section, the algorithm in our main real implementation source code "robot_run_gaits.cpp" will be analyzed. We attempted to use code from the previous year that was provided as a foundation, but

the code had many bugs out-of-the-box and was not easy to stand up. Therefore, we decided to implement a custom gait controller from scratch. The program is a low-level control of the robot. It initializes UDP communication, generates gait trajectories, and runs a main control loop at 500 Hz corresponding to 2ms timestep. Safety protections and power limits are implemented to prevent damage.

The motion plan consists of three phases to reach a stable standing position, followed by the trajectory tracking for the chosen gait. Phase 0 of the standing routine is responsible for opening the hips with PD gains of 40 and 3 for Kp and Kd respectively. Phase 1 executes the neutral pose to lean the robot back on its 'heels', and moves to intermediate hip/knee targets with Kp=50 and Kd between 3 and 4.5. Phase 2 is to raise the calves and knees into the final stand position. Stiffer gains are applied and then held. During the stand-up period (which lasts 10 seconds to ensure a stable position), RobotControl() calls MoveToStandNew() every tick and stores the initial yaw value from the IMU for later reference. This reference is only used by the walk gait to enable yaw-correction control.

Moving onto the gait execution, the generate_trajectory() function populates the gaitTrajs structure with per-leg joint arrays q1,q2,q3 for hip, thigh, knee over 100 time steps, for every possible gait. generate_trajectory() is the parent function that encapsulates the entire process outlined in Section VIII. This is executed prior to the robot making any action, so that during execution, the robot only needs to reference these values.

X. CONTROL SYSTEM

Instead of using force control to stabilize the robot and prevent drift, we used the robots built in IMU to get the roll pitch and yaw, from LowState.imu.rpy. The reason for this is that the main adjustment can be a result of the robot's heading, which is the yaw value. Using the standard_yaw value determined from the stand position, an error can be computed for each loop cycle between the current_yaw value, which is computed using a circular buffer of length 20, and the standard_yaw. This error is scaled by a proportional gain of 1.0, which is directly fed into the two front hip joints. If the robot lists to the left, the left hip will lean inwards while the right hip leans outwards. This allows the robot to constantly correct its yaw position as it continues to step, and walk in a straight line without user intervention.

During the execution of the stand sequence, linear interpolation is used between desired positions and the robot's current position. Before each phase of the process described in Section IX, the starting joint angles are saved. This is interpolated against the target joint angles over time, which achieves a very smooth motion.

During the gait execution, the robot reads from the gaitTrajs structure that was pre-loaded with the desired trajectories, in a constant loop. This allows the robot to consistently execute the same gait, over and over again. The robot uses PD control to move to these positions and support itself. The Kp values were 40 for the hip joints, 60 for the front shoulder/knee, and 90 for the rear shoulder and knee. Kd was set to 4 for all joints.

XI. SIMULATION IMPLEMENTATION

ROS Noetic is used to simulate the robot gaits on Gazebo. The simulation requires the following unitree packages to be installed on ros: unitree_ros and unitree_legged_sdk. The first step is to create a workspace where all the packages can be placed. After building the workspace, it should be correctly linked to the ROS environment (in other words, sourced). The structure of our simulation code consists of three header files, three C++ source files, and the corresponding CMakeLists file. The header files are placed in include folder under unitree_ros/ unitree_controller directory, while the source files are in the src folder of that. The content of CMakeLists.txt file of the same directory should be replaced with ours. The "walk_gait.cpp" file utilizes header files to manage the sensor data subscriptions for the walk gait,

and this can easily be switched for any other gait. The main function sets up ROS Publishers to send commands. After putting the robot into a safe standing-up position through motion_init() function, our custom function generate_trajectory() pre-calculates the trajectory of all gaits. In the control loop, warmup cycles are defined to have the robot executing the walk gait, followed by the chosen gait execution using forward_move() function.

The simulation results show that the robot model can successfully execute all four gait trajectories. Starting with the walk and trot gaits, the robot smoothly executes the gait commands and can continue without falling down. When it comes to the gallop and canter gaits, the robot fails to successfully transit from walk warm-up phase to the corresponding gait; however, the feet trajectories are executed as planned.

XII. REAL ROBOT IMPLEMENTATION

The real implementation of the robot gaits uses unitree_ros_to_real package. The package includes necessary library files to be able to control the robot at both high and low levels. The same build process from the simulation section should be followed. The header files utilized in real implementation are spline.h and custom gait.h. The first header file is included to generate gait trajectories. “robot_run_gaits.cpp” source file is placed in unitree_legged_sdk/example directory. The file is compiled: g++ robot_run_gaits.cpp -I ../include -L ../lib/cpp/amd64 \ -lunitree_legged_sdk -pthread -o robot_run_gaits. Given that the computer is connected to the robot’s wireless network, the cpp file is run.

From this, we were able to run and test each robot's gait. Staring with testing each gait while the robot is hanging to ensure the path of the foot trajectories. This, however, caused some issues in that the Kp and Kd were set very low based on the unitree examples. The lower Kp and Kd values unknowingly changed the foot trajectories to be shorter than the generated path. So, after verifying the foot has the correct trajectory and attempting to run on the ground, the robot would not be strong enough to resist its own weight and fall over. This is also a result of the weaker kp and kd values. This was fixed when we developed a stand up function from the unitree PD function that allowed for significantly higher Kp and Kd values without disrupting the q values from the generated gait trajectory.

This stand up function allowed us to test each gait after standing from the ground. This prevented problems such as placing the robot on the ground from hanging in an awkward state.

To walk in a straight line we started from stand up. From the stand up the robot takes a slight pause which is where the yaw heading of our IMU controller is created and averaged out. Then the walking gait and foot trajectories are cycled through as described in earlier sections. To walk in a straight line we had to tune the hip angle and direction to push the robot back to the standard yaw that we set. This was difficult and not the best way of implementing the controller. Rather, the yaw should dynamically change the foot trajectory by modifying the inverse kinematics for the hip joint to preserve the level of the body. This could be useful for future groups.

For the trot we use the same Kp and Kd for walk. The major difference is that we had to reduce the gait period in order to shorten the robots stride in the gait. This is because it was taking too large of steps and becoming wildly unbalanced. After this the robot was very stable and moved in the straightest path compared to the other gaits without the IMU controller.

For the gallop gait, some modifications were required from trot. First, the calf joint required an additional 30 Kp factor added to maintain the strength to support the robot. We also had to set the beta value (the percentage of time the robot is in stance) to 0.40 in order to maintain stability.

For the canter gait, the only modification needed was setting the hip joints from a typical 0 rad angle to 0.1 rads, which created a wider pose that created additional stability.

XIII. RESULTS

Our results were that we achieved several of the objectives that we set out to accomplish:

1. Gait design and implementation
    1.1. Designed from scratch gait generation code for all four gaits that allows a user to set the desired forward velocity, gait period, and swing height.
    1.2. Converted this codebase from MATLAB into C++ for generation on-board the robot.
    1.3. Successfully implemented all gaits in simulation.
    1.4. All gaits were successfully demonstrated by the physical robot in accordance with the swing height, phase shift, and swing time.
2. Control system
    2.1. A simple state machine was able to move from resting to standing to executing any gait
    2.2. UDP communication provided real-time feedback and updates to the robot.
    2.3. In the walk gait, a simple yaw-correction controller was implemented using the IMU to fix the list that the robot is prone to, and keep it walking in a straight line.
3. Real Robot implementation
    3.1. Designed and implemented each gait safely
    3.2. Verified wireless control
    3.3. Tested and verified the low state control operation
4. IMU heading control
    4.1. Implement IMU heading adjusting system.
    4.2. Achieved walking in a straight line
    4.3. Maintained stable movement for walking gait.

5. Challenges and Limitations

1. Converting gait generation code from Matlab to C++ was tedious and time-consuming due to a lack of familiarity with C++.
2. Gallop and canter gait simulations had instabilities that resulted in standing failures.
3. Previous year code had several challenges preventing the implementation which led to the implementation of our own code.
4. Tuning of the control parameters caused the progress to be slow due to the number of elements included in IMU control.

XIV. DELIVERABLES:

OURS GROUP-RESULTS (FALL 2025):

Testing gaits with sinusoidal trajectories (Only for testing purposes):

Sinusoidal-trajectories_gaits

Video documentation of our real robot results are found here:

EEL5669 Final Project Videos

All code for the simulations and hardware implementation is here:

https://github.com/twinjordan02/5669-Final-Project/tree/main

XV. CONCLUSION

This project successfully demonstrated the modeling, simulation, and real-world execution of four biologically inspired quadrupedal gaits such as walk, trot, canter, and gallop on the Unitree Go1 robot. In simulation, the gait planning and control pipeline was validated using ROS noetic and Gazebo, confirming correct timing, trajectory generation, and body stabilization before deployment on hardware. On the physical Go1, the same trajectory generator and low-level control structure were integrated into Unitree's SDK to run in real time. Through incremental testing starting with the robot suspended for safety, we achieved stable execution of all four gaits on flat

terrain. Additional improvements, such as an IMU-based yaw-correction controller, enabled the gaits to maintain a straight path without reliance on force control. Demonstrating the flexibility of the control architecture.

XVI. CONTRIBUTIONS

Abraham Alcala – Gaits and force feedback research and implementation, kinematics research and calculations, report writing. Assisted in simulation and hardware testing and execution.

Steven Booth – Co-wrote the gait planning and trajectory code first in MatLab, then helped rewrite in C++. Assisted with simulations and control implementation ideas. Wrote gait planning and trajectory sections for the report.

Marlon Garcia – Implementing the sinusoidal trajectories for testing purposes. Setting up communication protocols with the robot through Wi-Fi and Ethernet. Helping with the physical and coding implementation of the stand phase, and IK code. Besides, testing ROS packages and CMakeList files with the physical robot. After a lot of trials and testing (with previous semester's files), I arrive at the conclusion that it is better to only execute C++ code in low state, instead of using the previous semester packages. Assisting with the IMU coding and trials with the physical implementation. Assisting with the force control simulation for the gallop gait.

Thomas Greenwood - Assisted in research of force feedback and gaits.

Jordan Haag - Co-wrote the gait planning and trajectory code in Matlab, then the rewrite to C++. Assisted in simulation development and validation of joint trajectories. Assisted in the yaw control development for the walk gait on the physical robot, and the overall control loop for gait control. Developed initial kinematics math and assisted in report writing.

Tristan Haack - Researched force feedback and IMU feedback methods, developed kinematic calculations, implemented hardware development and integration with the physical robot. Assisted in the testing of the sinusoidals trajectories. Also tuned the many gait feedback loops and Kp/KD parameters. Reporting writing.

Nihad Habizada - Assisted in the ROS Installation, simulation results, and the physical implementation of the robot's gaits. Helped with testing the sinusoidals trajectories and the IK codes. Contributed to the report writing.

XVII. CITATIONS

[1] Dong, S., Fan, F., Chen, Y., Guo, S., and Liu, J., "Gait planning, and Motion Control Methods for quadruped robots: Achieving high environmental adaptability: Review," *Computer Modeling in Engineering & Sciences*, vol. 143, Apr. 2025, pp. 1–50.

[2] Li, Z., Song, Y., Zhang, X., Peng, X., and Xu, N., "Modeling of walking-gait parameters and walking strategy for quadruped robots," *Applied Sciences*, vol. 13, Jun. 2023, p. 6876.

[3] Hao, Q., Wang, Z., Wang, J., and Chen, G., "Stability-guaranteed and high terrain adaptability static gait for Quadruped Robots," *Sensors*, vol. 20, Aug. 2020, p. 4911.

[4] M. H. Raibert, *Legged robots that balance*. London: Mit P, 2000.

[5] Craig, J. J., Introduction to Robotics: Mechanics and Control, 3rd ed. Pearson Education India, 2009.